Barber System

Documentação Técnica Completa do Sistema

Este documento explica em detalhes como o sistema funciona, suas configurações, lógicas de negócio, segurança (JWT + Spring Security) e como testar corretamente no Postman. Use-o como referência para continuar o desenvolvimento sem gerar erros inesperados.

1. Stack e Configurações Principais

O projeto é um backend **Spring Boot** (Java 21) focado em gerenciamento de barbearia, com autenticação **JWT stateless** e Spring Security.

Item	Valor
Spring Boot	4.1.1 (parent)
Java	21
Banco de Dados	MySQL (jdbc:mysql://localhost:3306/barberShop)
Usuário / Senha	root / (vazio)
ddl-auto	update
JWT Secret	jwt.secret (variável de ambiente ou valor padrão longo)
JWT Expiration	7200000 ms (2 horas)
Password Encoder	BCrypt

Arquivo principal de configuração: **src/main/resources/application.properties**

2. Arquitetura e Papéis (Roles)

Existem **3 tipos de usuário**, todos autenticados pelo mesmo **CustomUserDetailsService**:

- **ADMINISTRADOR**
- **BARBEIRO**
- **CLIENTE**

O serviço tenta buscar o e-mail nesta ordem: **Administrador** → **Barbeiro** → **Cliente**.

Cada entidade possui os campos **nivelAcesso** (enum) e **status** (ATIVO / INATIVO). Usuários com status INATIVO ficam com `.disabled(true)` no Spring Security.

3. Segurança (JWT + Spring Security)

3.1 Configuração de Segurança (SecurityConfiguracao.java)

- CSRF desabilitado (aplicação stateless)
- Sessão: **STATELESS**
- `/login` → público (permitAll)
- `POST /barbeiros/**` → somente **ROLE_ADMINISTRADOR**
- Qualquer outra rota → autenticado

3.2 Fluxo JWT

1. Cliente faz **POST /login** com JSON `{ "email": "...", "senha": "..." }`
2. Spring autentica via **AuthenticationManager** + **CustomUserDetailsService**
3. **JwtService** gera token contendo: `subject` = e-mail e `roles` = lista de roles
4. Retorna `{ "token": "eyJ...", "tipo": "Bearer" }`
5. Nas próximas requisições o cliente envia: `Authorization: Bearer eyJ...`
6. **JwtAuthFilter** intercepta, valida o token e popula o **SecurityContext**

4. Endpoints Atuais (Barbeiro)

Base path: `/barbeiros`

Método	URL	Quem pode	Descrição
POST	/barbeiros	Só ADMINISTRADOR	Cadastrar barbeiro
GET	/barbeiros/lista/ativos	Autenticado	Lista ativos
GET	/barbeiros/lista/inativos	Autenticado	Lista inativos
GET	/barbeiros?nome=joao	Autenticado	Busca por nome (containing + ignoreCase)
GET	/barbeiros/{id}	Autenticado	Busca por ID
PATCH	/barbeiros/{id}	Autenticado	Atualização parcial

5. Lógicas de Negócio Importantes (BarbeiroService)

5.1 Cadastro de Barbeiro (cadastroBarbeiro)

1. Verifica se e-mail já existe → **DuplicateResourceException** (HTTP 409)
2. Verifica se telefone já existe → HTTP 409
3. Verifica se `senha == confirmaSenha` → **BadRequestException** (HTTP 400)
4. Mapeia DTO → Entity (sem senha)
5. Criptografa a senha com BCrypt **no Service** (nunca no Mapper)
6. Define `nivelAcesso = BARBEIRO` e `status = ATIVO`
7. Salva e retorna **BarbeiroSaidaDto** (sem senha)

5.2 Atualização Parcial (PATCH)

- Só atualiza os campos que vierem não-nulos no **BarbeiroAtualizaDto**
- Valida unicidade de e-mail/telefone ignorando o próprio ID

6. Padrão de Mapper (Fundamental)

Sempre siga esta estrutura para evitar erros e manter o código limpo:

`toEntity(DtoEntrada)` → somente criação
`toSaidaDto(Entity)` → resposta da API
`atualizarEntity(DtoUpdate, Entity)` → update parcial
`toSaidaDtoList(List<Entity>)` → listas

Regra de ouro: Nunca criptografe senha no Mapper. Sempre faça isso no Service.

7. Tratamento Global de Erros

Exceção / Situação	HTTP	Resposta
Credenciais inválidas (login)	401	"Email ou senha inválidos"
Sem permissão (role)	403	"Você não tem permissão para esta ação"
Validação @Valid (DTO)	400	Mapa de campos com mensagens
ResourceNotFoundException	404	Mensagem da exceção
DuplicateResourceException	409	Mensagem da exceção
BadRequestException	400	Mensagem da exceção

8. Como Testar no Postman (Versão Atualizada)

> **Atenção:** O documento .docx existente na pasta Documentação está **desatualizado** (fala de sessão + form-urlencoded). O sistema atual usa **JWT + JSON**.

8.1 Preparação

1. Suba o MySQL e crie o banco:

```
CREATE DATABASE barberShop;
```

2. Crie um Administrador manualmente no banco (ainda não existe endpoint de cadastro de admin). A senha precisa estar criptografada com BCrypt.

8.2 Login

- Método: **POST**
- URL: `http://localhost:8080/login`
- Headers: `Content-Type: application/json`
- Body (raw → JSON):

```
{ "email": "admin@gmail.com", "senha": "123456" }
```

Resposta esperada:

```
{ "token": "eyJhbGciOiJIUzI1NiJ9...", "tipo": "Bearer" }
```

8.3 Requisições Autenticadas

Copie o token e, nas próximas requisições, use a aba **Authorization** → Type: **Bearer Token** ou adicione o header manualmente:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

8.4 Exemplos de Testes

Cadastrar barbeiro (precisa ser Admin):

```
POST http://localhost:8080/barbeiros
{
  "nome": "João Silva",
  "dataNascimento": "1990-05-15",
  "telefone": "(11) 98765-4321",
  "especialidade": "Corte e Barba",
  "email": "joao@email.com",
  "senha": "Senha@123",
  "confirmaSenha": "Senha@123"
}
```

Listar ativos:

```
GET http://localhost:8080/barbeiros/lista/ativos
```

Buscar por nome:

```
GET http://localhost:8080/barbeiros?nome=joao
```

Atualizar parcialmente:

```
PATCH http://localhost:8080/barbeiros/1
{ "especialidade": "Corte Premium" }
```

9. Regras para Continuar Codando sem Erros

1. **Sempre use o PasswordEncoder** no Service ao criar ou alterar senha.
2. Respeite a ordem de busca no **CustomUserDetailsService** (Admin → Barbeiro → Cliente).
3. Novos endpoints protegidos: use `@PreAuthorize("hasRole('ADMINISTRADOR')")` ou configure no SecurityFilterChain.
4. DTOs de saída **nunca** devem conter senha.
5. Use **PATCH** para updates parciais (padrão já estabelecido).
6. Exclusão deve ser **lógica** (mudar status para INATIVO), nunca física.
7. Siga o padrão de Mapper: toEntity / toSaidaDto / atualizarEntity.

O que ainda falta implementar:

- Controllers e Services completos de Cliente e Administrador
- Alterar senha
- Reativar / Afastar barbeiro
- Módulo de Agendamento
- Endpoint de cadastro de Administrador (seed inicial)